

ALPR Sesión 1

CHALLENGE #1 · PSIV2 VISION & LEARNING · UAB · 18 SEPTIEMBRE 2026

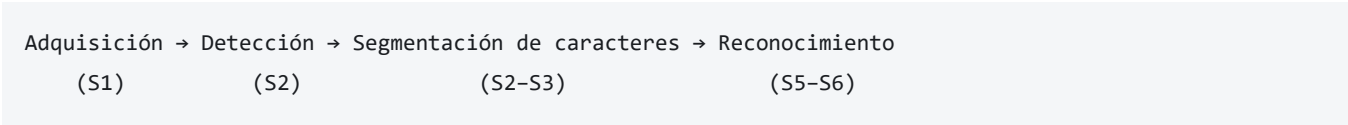
Informe completo del trabajo hecho: qué se implementó, por qué, qué hace cada función del código y qué números tenemos para defender la sesión.

1. Dónde estamos	7. Los scripts
2. Cómo se llegó hasta aquí	8. main.py y los tests
3. El dataset	9. Las tres métricas
4. config.py	10. La anotación de las 200 cajas
5. alpr/detection.py	11. Qué queda pendiente
6. alpr/dataset.py	12. Preguntas probables en la revisión

1. Dónde estamos

La Sesión 1 está cerrada salvo dos condiciones de dataset: tenemos detector propio, 207 imágenes con ground truth, 200 cajas anotadas a mano y tres métricas medidas.

El challenge es reconocer matrículas de coches a partir de fotos. Se descompone en cuatro etapas, y cada sesión del calendario ataca una:



Nosotros estamos al final de la etapa 1 y con la 2 preparada. El profesorado dio un script de exploración de datos que **no arrancaba**: importaba un módulo `license_plate` que no existía, y apuntaba a `D:\Teaching\...`, una ruta de su propio ordenador. Implementar ese módulo desde cero fue el grueso del trabajo.

LOS SEIS OBJETIVOS DEL ENUNCIADO

OBJ	QUÉ PIDE	ESTADO
1	Impacto de cómo se adquieren los datos	Hecho — demostrado con estadística (§9)
2	Morfología en pre y post-proceso	Hecho — es el detector entero (§5)
3	YOLO	Preparado: dataset exportado y línea base medida
4	Estructuras de imagen (bordes, blobs)	Hecho — Sobel y contornos en el detector
5	Descriptores (HOG, LBP)	Sesión 5
6	Split simple vs. cross-validation	Split estratificado hecho, semilla fija

Restricción del enunciado: nada de Jupyter Notebooks. Todo en `.py` (el enunciado especifica Spyder + numpy/pandas/sklearn/PyTorch).

2. Cómo se llegó hasta aquí

El trabajo siguió este orden, y cada paso salió de un problema concreto del anterior.

1. **Arreglar lo que no arrancaba.** Implementar `detectPlates()` desde cero y sustituir las rutas del profesor por rutas relativas al repo.
2. **Descubrir que el dataset tenía 29 duplicados exactos.** Detectados por MD5. Sin deduplicar, toda la estadística salía sesgada.
3. **Caracterizar el protocolo de adquisición leyendo los EXIF.** Las 69 imágenes del profesorado son del mismo móvil, mismo tipo de día y mismo parking.
4. **Demostrar el Obj1 con estadística.** El ángulo separa Frontal de Lateral; el color y la iluminación no separan nada, porque todas las fotos comparten cámara y escenario.
5. **Ampliar el dataset con 138 fotos propias**, en dos lotes y con dos móviles distintos. Aquí la iluminación sí separa los dos datasets, y con eso el Obj1 queda demostrado de verdad.
6. **Medir el acierto real revisando los 200 recortes a mano.** Salió 67%, no el 99% que parecía.
7. **Anotar las 200 cajas** y medir el IoU del detector contra ellas.

Tres decisiones que hay que saber explicar

WORK_WIDTH = 400 , no 1024. El detector reduce la imagen antes de trabajar. La receta clásica usa ~1024 px, pero aquí el coche llena el encuadre y la matrícula ocupa el 23% del ancho. Los kernels morfológicos actúan a escala de *carácter*, así que lo que importa es cuántos píxeles mide una letra, no la imagen. Medido sobre 12 imágenes: a 400 px acierta 10 de 11, a 1024 px solo 6 de 12, y además el recorte parte la matrícula por la mitad.

El ángulo se calcula a mano, no con el código del profesor. `cv2.minAreaRect` devuelve el ángulo en un rango que **cambia entre versiones de OpenCV** (`[-90,0)` en la 5.0, `(0,90]` en las 4.x). La corrección del código base (`angle - 90 if w < h`) daba **-180° para una placa horizontal**. Hay un test que lo cubre.

Nunca inventar una matrícula para esquivar una colisión de nombres. El dataset original tenía dos fotos del mismo coche; como no se puede repetir nombre de fichero, alguien cambió un dígito del nombre. Eso rompe el ground truth en silencio. La solución es el sufijo `_2` , que el lector de matrículas ignora porque su expresión regular está anclada al inicio del nombre.

3. El dataset

El nombre del fichero ES el ground truth. Un fichero llamado con la matrícula significa que en esa foto se ve esa matrícula. Eso da etiquetas gratis para evaluar las etapas 2 a 4, y es la convención que hay que mantener al añadir fotos.

CARPETA	FRONTAL	LATERAL	OTROS	TOTAL
real_plates (profesorado)	19	50	—	69
new_plates (nuestras)	103	30	5	138
Total	122	80	5	207

new_plates/OtrosFormatos/ guarda 5 matrículas que no siguen el formato español moderno: 2 francesas, 1 andorrana, 1 británica y 1 española antigua. Queda fuera de la estadística pero documentada, porque es un caso real que un ALPR desplegado se encuentra.

Las 29 duplicadas

El dataset del profesorado traía 98 ficheros, pero **29 eran copias byte a byte**: los `PXL_2021*.jpg` son el mismo fichero que los ya nombrados con la matrícula. Quedan 69 únicas. Cualquier media calculada sin deduplicar cuenta esas imágenes dos veces.

Se detectan comparando el hash MD5 del contenido, no el nombre. Ya están borradas del disco, pero el código sigue deduplicando al vuelo por si reaparecen.

El protocolo de adquisición, medido de los EXIF

Constante en las 69: **Google Pixel 4 XL**, focal 4.38 mm, f/1.73, 4032×2268, **sin flash**. Seis días entre el 21 de septiembre y el 8 de octubre de 2021. **ISO 39–1809, mediana 480**: ISO alto significa parking cubierto con luz artificial, que es donde está tomada la mayoría.

Consecuencia directa: **color, saturación e iluminación NO separan Frontal de Lateral** ($p > 0.7$ en los tres), porque todas las fotos comparten cámara y escenario. Lo único que separa las vistas es el ángulo. Por eso el enunciado pide ampliar el dataset.

Errores de etiqueta que encontramos

Leer 138 matrículas a mano tiene tasa de error. Se detectaron siete, por tres vías distintas:

CÓMO SE DETECTÓ	CUÁNTOS	QUÉ PASÓ
Validador de formato	2	Se había leído una O y una Q, letras que no existen en matrículas españolas
Revisión manual	4	Dígitos y consonantes confundidos a baja resolución
Colisión de nombres	1	Dos fotos del mismo coche; se renombró una inventando un dígito

El formato español moderno es **4 dígitos + 3 consonantes**, sin vocales y sin Ñ ni Q. Esa regla es la que permite validar automáticamente.

La lección que hay que saber contar: una matrícula se leyó mal (un 8 como una B) y por eso pareció de formato antiguo, así que se clasificó a mano como excepción. Al tratarla como excepción **se saltó el validador de formato**, que es justo lo que la habría cazado. De ahí que ahora haya tests automáticos de integridad del ground truth.

Tasa de error conocida del etiquetado: **3 de 108 ($\approx 3\%$)** en el segundo lote. Es una cota inferior, porque solo cuenta los errores que alguien llegó a revisar. Conviene tenerlo presente al interpretar la precisión del OCR en las Sesiones 5-6: parte del error medido será de las etiquetas, no del modelo.

Privacidad

Las fotos **no se versionan**. Son matrículas reales de terceros, o sea dato personal identificable (RGPD). El repositorio de GitHub es privado. Sí se versionan las anotaciones — `metadata.csv` , los veredictos, las etiquetas YOLO — porque son nuestro trabajo y pesan poco.

4. config.py

Rutas y constantes compartidas. Existe para que el proyecto funcione en cualquier máquina: el código base tenía `D:\Teaching\...` escrito a fuego.

```
ROOT = Path(__file__).resolve().parent
REAL_PLATES_DIR = ROOT / "real_plates" # dataset del profesorado
NEW_PLATES_DIR = ROOT / "new_plates" # dataset ampliado por nosotros
OUT_DIR = ROOT / "out" # figuras y CSV (no versionado)
VIEWS = ["Frontal", "Lateral"]
PLATE_AR = 520 / 110 # ~4.73
```

`PLATE_AR` es la relación de aspecto real de una matrícula española (520×110 mm). Se usa como referencia al juzgar si una caja es plausible. `VIEWS` no incluye `OtrosFormatos`, y por eso esa carpeta queda fuera de toda la estadística automáticamente.

5. alpr/detection.py

El corazón del proyecto. Sustituye al módulo `license_plate.LicensePlateDetector` que el código base importaba y que no venía con el enunciado.

La idea: una matrícula es **caracteres oscuros sobre chapa clara**, lo que produce una firma muy característica — bordes verticales densos y regulares dentro de una región brillante. Todo el detector busca esa firma.

CONSTANTES

CONSTANTE	VALOR	POR QUÉ
<code>WORK_WIDTH</code>	400	Ancho al que se reduce la imagen. Medido, no elegido (§2)
<code>AR_RANGE</code>	(2.5, 6.5)	Aspecto admisible. 4.73 de frente, menos en lateral por perspectiva
<code>AREA_FRAC_RANGE</code>	(0.001, 0.08)	Área del candidato sobre el área de la imagen. Una matrícula ocupa ~0.023
<code>SEARCH_TOP</code>	15	Contornos más grandes que se examinan
<code>MAX_CANDIDATES</code>	10	Candidatos que se devuelven como mucho

Las funciones

`normalized_angle(rect) → float`

Ángulo del **lado largo** del rectángulo respecto a la horizontal, en el rango $(-90, 90]$. Cero grados significa lado largo horizontal, o sea vista frontal.

Existe porque `cv2.minAreaRect` no garantiza que el lado que llama *width* sea el mayor, así que su ángulo unas veces se mide sobre el lado largo y otras sobre el corto. Aquí se calcula el lado largo con `atan2` sobre los cuatro vértices, lo que **no depende de la versión de OpenCV**. Después se pliega al rango, porque la orientación es módulo 180° .

`aspect_ratio(rect) → float`

Lado largo dividido entre lado corto. Trivial pero está aparte para que sea independiente de cuál de los dos lados llamó *width* OpenCV.

`plate_mask(gray, steps=None)` → máscara binaria

El pipeline morfológico completo, en seis pasos. **Esto es el Obj2 del enunciado.** Si se le pasa un diccionario en `steps`, guarda ahí cada imagen intermedia — así el script que ilustra el pipeline no reimplementa nada y nunca se desincroniza.

#	OPERACIÓN	PARA QUÉ
1	Escala de grises	Punto de partida
2	Blackhat , kernel 13×5	Resalta lo oscuro sobre fondo claro: los caracteres
3	Cierre 3×3 + Otsu	Máscara de zonas claras: aísla la chapa blanca
4	Sobel-x + normalizar	Bordes verticales: los caracteres los generan densos y regulares
5	Gauss 5×5 + cierre 13×5 + Otsu	Une los huecos entre caracteres para que la matrícula sea un solo blob
6	Erosión ×2, dilatación ×2, AND con (3), dilatación ×2, erosión ×1	Limpia ruido y se queda solo con lo que cae en zona clara

El kernel 13×5 es rectangular y ancho a propósito: une caracteres vecinos horizontalmente sin unir la matrícula con lo que tiene encima o debajo.

`candidate_verdict(rect, image_area, ...)` → (bool, motivo)

El único sitio donde vive el criterio de aceptación. Devuelve si el rectángulo pasa los filtros y, si no, por qué no. Comprueba dos cosas:

1. **Área** como fracción de la imagen, dentro de `AREA_FRAC_RANGE`. Descarta ruido por abajo y carrocería por arriba.
2. **Relación de aspecto**, dentro de `AR_RANGE`.

Que devuelva el motivo es lo que permite al script explicativo construir una tabla de decisión contorno a contorno. Y que esté centralizado significa que si cambias el criterio, cambian a la vez el detector y su explicación.

`detectPlates(image, ...)` → lista de contornos

La función que pedía el código base. Devuelve contornos aptos para `cv2.minAreaRect`, ordenados de mayor a menor área, o sea de mayor a menor plausibilidad. Los pasos:

1. Reducir la imagen a `WORK_WIDTH` y guardar el factor de escala.
2. Pasar a gris y aplicar `plate_mask`.
3. `findContours` sobre la máscara y ordenar por área descendente, quedándose con los `SEARCH_TOP` mayores.
4. Para cada uno, `minAreaRect` y `candidate_verdict`.
5. **Reescalar los contornos aceptados a la imagen original** antes de devolverlos.

El paso 5 es importante y fácil de olvidar: **el ángulo es invariante a la escala, pero el área no.** Si devolvieras los contornos en coordenadas de la imagen reducida, cualquier medida de área posterior estaría mal por un factor de $(4032/400)^2 \approx 100$.

6. alpr/dataset.py

Carga del dataset, deduplicado y dibujo de candidatos.

`file_md5(path) → str`

Hash MD5 del contenido del fichero. Sirve para detectar duplicados exactos: dos ficheros con distinto nombre pero el mismo contenido dan el mismo hash.

`plate_from_filename(path) → str | None`

Extrae la matrícula del nombre, o `None` si el nombre no la lleva. Usa la expresión regular `^(\d{4}[A-Z]{3})`.

El detalle clave es el `^`: está anclada al *inicio* del nombre, así que ignora cualquier sufijo. Un fichero terminado en `_2` o en `- copia` sigue devolviendo la matrícula correcta. Eso es lo que permite tener varias fotos del mismo coche en la misma carpeta sin perder el ground truth, y lo que hace innecesario inventar matrículas para esquivar colisiones.

`unique_images(directory) → (lista de rutas, nº descartados)`

Imágenes de la carpeta sin duplicados exactos, más cuántos se descartaron.

Ante un duplicado **conserva la copia cuyo nombre lleva la matrícula**, que es la que tiene ground truth.

Lo consigue ordenando primero por «¿tiene matrícula en el nombre?» antes de recorrer, de modo que esa copia gana siempre el desempate del `setdefault`.

`draw_candidates(image, contours, angles=None, color=...) → imagen`

Dibuja el `minAreaRect` de cada candidato, anotado con su ángulo. Es el tercer bullet de la slide 16 del enunciado, que el código base no hacía. El grosor del trazo se escala con el ancho de la imagen para que se vea igual en fotos de cualquier resolución.

7. Los scripts

Uno por entregable, numerados por orden de ejecución. Cada uno es ejecutable por separado, porque el enunciado evalúa los entregables uno a uno.

SCRIPT	QUÉ HACE
<code>00_acquisition_protocol.py</code>	Lee los EXIF y caracteriza el protocolo de adquisición: cámara, focal, apertura, ISO, flash, fechas. Es el segundo bullet de la slide 16.
<code>01_data_exploration.py</code>	Reescritura del script del profesorado. Deduplica por MD5, convierte a HSV una sola vez (el original lo hacía tres veces), compara distribuciones por vista y por dataset con Mann-Whitney U, y vuelca <code>out/properties.csv</code> .
<code>02_show_detections.py</code>	Dibuja las cajas candidatas sobre cada imagen y además guarda recortes individuales del candidato #1, más un índice <code>candidatos.csv</code> . Esos recortes son los que se revisaron a mano para medir el acierto.
<code>03_explain_pipeline.py</code>	Material para la memoria: panel con las 6 etapas de <code>plate_mask()</code> y tabla de decisión contorno a contorno. Reutiliza <code>plate_mask(steps=...)</code> y <code>candidate_verdict()</code> , así que nunca miente sobre lo que hace el detector real.
<code>05_detection_accuracy.py</code>	Calcula la tasa de acierto real a partir de los veredictos revisados a mano, con intervalos de confianza de Wilson al 95%.
<code>06_export_yolo.py</code>	Exporta el dataset en formato ultralytics. Usa los veredictos como pre-anotación : escribe la caja del detector donde acierta y deja vacías las demás. Split estratificado por dataset y vista, semilla fija 42 (160 train / 40 val). Nunca pisa una etiqueta existente sin <code>--force</code> .
<code>07_annotate.py</code>	Anotador interactivo. Ver §10.
<code>08_iou_baseline.py</code>	Mide el IoU del detector contra las cajas anotadas. Ver §9.
<code>clean_real_plates.py</code>	Deja <code>real_plates/</code> consistente: borra duplicados y renombra a <code><MATRÍCULA>.jpg</code> . Es <i>dry-run</i> por defecto; hace falta <code>--apply</code> .

8. `main.py` y los tests

`main.py` encadena las etapas, corta si alguna falla y resume tiempos al final. Las etapas viven en una lista `STAGES`; al añadir una sesión nueva se amplía ahí.

```
python main.py           # pipeline completo
python main.py --list    # qué etapas hay
python main.py explore detect  # solo esas etapas
python main.py --new      # incluir new_plates/ en la exploración
```

Lanza cada etapa como subprocesso, no la importa. Es a propósito: los scripts empiezan por dígitos, así que no son importables como módulos de Python, y además así siguen siendo ejecutables por separado.

Los dos ficheros de tests

FICHERO	QUÉ FIJA
<code>tests/test_detection.py</code>	5 tests: recuperación del ángulo, que una placa horizontal dé 0°, que el aspecto no dependa de la orientación, detección sobre una placa sintética, y recuperación del ángulo tras rotar.
<code>tests/test_dataset.py</code>	5 tests de integridad del ground truth : formato de todos los nombres, que nada en <code>OtrosFormatos</code> sea en realidad una matrícula española válida, que <code>metadata.csv</code> y el disco coincidan, y que no queden imágenes sin ground truth.

`test_dataset.py` nació de un error real: una matrícula mal leída se clasificó a mano como excepción y así esquivó el validador. Los tests existen para que esa clase de error no pueda repetirse en silencio.

9. Las tres métricas

Este es el punto donde más fácil es equivocarse al explicar el proyecto. Hay **tres números distintos** y miden cosas distintas.

MÉTRICA	VALOR	QUÉ MIDE DE VERDAD
Cobertura	99.0%	En cuántas imágenes el detector devolvió <i>algo</i> . Un faro cuenta igual.
Acierto	67.0%	En cuántas el recorte contenía la matrícula legible. Revisado a mano, una a una.
IoU	0.562	Cuánto se solapa la caja propuesta con la real. Lo único que mide la localización.

Cobertura no es acierto. El 99% dice que casi siempre encuentra *algún* candidato, con mediana de 3 por imagen. Como la mayoría no son la matrícula, ese número solo no significa nada. Si en la revisión alguien dice «99% de detección», hay que corregirlo.

El acierto, por grupos

GRUPO	ACIERTO	IC 95%
TOTAL	67.0% (134/200)	60.2 – 73.1%
real_plates	86.8% (59/68)	76.7 – 92.9%
new_plates	56.8% (75/132)	48.3 – 65.0%
new_plates Lateral	43.3% (13/30)	27.4 – 60.8%

El IoU, por grupos

GRUPO	IOU MEDIO	MEDIANA	≥ 0.5	≥ 0.7
TOTAL	0.562	0.678	65.0%	48.0%
real_plates	0.747	0.806	85.3%	70.6%
new_plates	0.467	0.600	54.5%	36.4%
Frontal	0.560	0.676	63.6%	46.3%
Lateral	0.565	0.709	67.1%	50.6%

El dato que más confianza da al trabajo: el 65.0% a $\text{IoU} \geq 0.5$ coincide con el 67.0% de acierto revisado a ojo. Son dos medidas *independientes* — una geométrica y automática, otra humana y cualitativa — que se diferencian en 2 puntos. Eso valida las dos a la vez.

El estándar habitual (PASCAL VOC, el $\text{mAP}@0.5$ que publica YOLO) acepta $\text{IoU} \geq 0.5$. Para ALPR conviene ser más estricto y pedir **$\text{IoU} \geq 0.7$ con la matrícula entera dentro**, porque detrás viene segmentar caracteres y una caja que corta la placa arruina esa etapa aunque solape medio. A ese umbral el detector baja al **48%**.

Localizar mal y cortar son fallos distintos

Sobre las 97 imágenes con cuadrilátero anotado, separando ambos fallos:

SITUACIÓN	%
Ni localiza la placa (IoU < 0.5)	32.0%
De las que sí localiza: la coge entera	solo 21.2%
De las que sí localiza: la corta	78.8%

Contención mediana 0.945, o sea que cuando corta se deja alrededor de un 5% de la placa: típicamente un carácter del borde. **Eso es lo que rompe la segmentación posterior**, más que los fallos de localización.

El Obj1, demostrado

Con `--new`, el script 01 compara protocolo contra dataset ampliado:

PROPIEDAD	REAL_PLATES	NEW_PLATES	P	VEREDICTO
Iluminación (V)	108.2	143.1	1.4e-26	SEPARA
Saturación (S)	41.8	49.3	2.0e-06	SEPARA
Área de placa	219432	173956	2.9e-02	SEPARA
Ángulo	4.2°	2.2°	0.14	no separa

Las nuestras son mucho más brillantes y saturadas, y tomadas desde más lejos: exterior soleado frente a parking cubierto. **Ese contraste medido es el argumento del Obj1**, y la caída de 87% a 57% en acierto es su consecuencia práctica: el detector morfológico **no generaliza**.

10. La anotación de las 200 cajas

Para entrenar YOLO y para medir IoU hace falta saber dónde está *de verdad* cada matrícula. Eso se anotó a mano con `07_annotate.py`.

CÓMO SE MARCÓ	CUÁNTAS
<code>quad</code> — marcando las 4 esquinas	97
<code>drawn</code> — rectángulo arrastrado	39
<code>approved</code> — pre-anotación del detector aceptada	64
Total	200

Por qué esquinas y no un rectángulo

La caja que pide YOLO es *axis-aligned*: sus lados son paralelos a los ejes. Sobre una matrícula inclinada, esa caja **tiene que** meter parachoques. Medido sobre las 97 anotadas:

MEDIDA	VALOR
AR real de la placa (mediana)	4.23
Inclinación (mediana / máx)	6.3° / 25.8°
Fondo extra que mete la caja recta	+77%

De ese +77%, unos 60 puntos son la inclinación (inevitable) y el resto el margen fijo del 4% que añadimos a propósito.

La solución fue marcar **las 4 esquinas de la placa**. De ahí el script deriva la caja recta añadiendo un margen fijo, igual para todas. Eso separa dos cosas que antes iban mezcladas:

- el **cuadrilátero** (`yolo_dataset/quads.csv`) es la placa exacta, sin fondo;
- la **caja** es un derivado reproducible, que se puede regenerar entera con otro margen sin reanotar nada.

El cuadrilátero no es solo precisión: es exactamente lo que hace falta en la Sesión 2 para `cv2.getPerspectiveTransform`, o sea rectificar la placa antes de segmentar caracteres. Las esquinas se ordenan solas a TL-TR-BR-BL, así que el cuadrilátero sirve tal cual. **El trabajo de anotar vale para dos etapas.**

Verificación de la anotación

Las 97 rectifican correctamente con `warpPerspective`, y las 97 placas resultantes **se leen y coinciden con el nombre del fichero**. Son 97 etiquetas de ground truth confirmadas de forma independiente, casi la mitad del dataset.

Además se comprobó automáticamente que ningún cuadrilátero tuviera las esquinas mal puestas: lados opuestos coherentes, aspecto plausible e inclinación razonable. Ninguno falló.

DETALLE DE USO

```
python scripts/07_annotate.py          # las no revisadas
python scripts/07_annotate.py --only-pending # solo las que no tienen caja
python scripts/07_annotate.py --review    # repasar las ya guardadas
python scripts/07_annotate.py --all      # las 200

c  marcar 4 esquinas      a  aprobar (GUARDA)
d  rectángulo arrastrado  s  saltar (NO guarda, vuelve a salir)
n  sin matrícula visible  z  deshacer  +/- zoom    q  salir
```

Guarda tras cada decisión, así que se puede parar y continuar. Las revisadas no vuelven a salir, de modo que al añadir fotos nuevas solo pide las que faltan. Van ordenadas **de más a menos sospechosa** según el aspecto de la caja.

11. Qué queda pendiente

Nada de código. Faltan **dos condiciones de dataset** que el enunciado nombra explícitamente:

CONDICIÓN	ESTADO
Luz solar directa · sombra · luz artificial	Cubierto
Reflejos, bajo contraste (carrocerías oscuras)	Cubierto
Viewpoints y cámaras distintas (3 móviles)	Cubierto
Matrículas traseras	Cubierto
Noche	0 imágenes
Desenfocadas	0 imágenes

Con 10–15 de cada bastaría. Conviene hacerlas **antes** de entrenar YOLO: si se entrena sin ellas, el modelo no las verá nunca y habría que repetir el ciclo.

Dos avisos menores para interpretar los resultados:

- El reparto Frontal/Lateral global es 122/80, o sea **60/40**. Aceptable, pero al evaluar por vista conviene usar métricas que no se dejen arrastrar por el desequilibrio.
- La columna `glare` del metadata solo tiene 3 casos marcados de 138. No da para estratificar: mejor tratarla como anotación descriptiva.

Para la Sesión 2

Está todo listo: dataset en formato ultralytics, split estratificado con semilla fija y línea base medida contra la que comparar.

Una decisión a tomar: **con 202 imágenes no se entrena YOLO de cero**. Toca *fine-tuning* de un modelo preentrenado (`yolov8n` o `yolov8s`).

12. Preguntas probables en la revisión

¿Qué detección usáis, clásica o deep learning?

Clásica, morfológica. Es lo que pide el Obj2 y es la línea base contra la que compararemos YOLO en la Sesión 2. Está medida: IoU medio 0.562, 65% a $\text{IoU} \geq 0.5$.

¿Cómo sabéis que el detector funciona?

Con tres medidas distintas, y sabiendo lo que mide cada una. Cobertura 99% (devuelve algo), acierto 67% (revisado a mano recorte a recorte) e IoU 0.562 (solape geométrico contra 200 cajas anotadas). El 65% a $\text{IoU} \geq 0.5$ y el 67% de acierto coinciden, y son medidas independientes.

¿Por qué reducís a 400 px y no a 1024?

Porque los kernels morfológicos actúan a escala de carácter, no de imagen. En este dataset el coche llena el encuadre. Está medido sobre 12 imágenes: 400 px da 10 de 11, 1024 px da 6 de 12.

¿Cómo demostráis el impacto de la adquisición (Obj1)?

Con Mann-Whitney U. Dentro del dataset del profesorado, el color y la iluminación *no* separan Frontal de Lateral ($p > 0.7$) porque todas las fotos comparten cámara y parking; lo único que separa es el ángulo. Al comparar con nuestras fotos, la iluminación separa a $p = 1.4e-26$. Y la consecuencia práctica es la caída de acierto del 87% al 57%.

¿Por qué el ángulo lo calculáis vosotros?

Porque `cv2.minAreaRect` cambia de convención entre versiones de OpenCV. La corrección del código base da -180° para una placa horizontal en OpenCV 5.0. Lo calculamos con `atan2` sobre los vértices, que no depende de la versión, y hay un test que lo fija.

¿Cuántas imágenes tenéis y de dónde salen?

207: 69 del profesorado (98 ficheros menos 29 duplicados exactos detectados por MD5) y 138 nuestras, tomadas con dos móviles distintos en dos lotes.

¿Qué fiabilidad tienen vuestras etiquetas?

Tasa de error conocida $\approx 3\%$ en el segundo lote, y es una cota inferior. Hay un validador de formato (4 dígitos + 3 consonantes, sin vocales ni Ñ/Q) que cazó dos errores reales, y tests automáticos de integridad del ground truth.

¿Por qué el detector falla más en vuestras fotos?

Porque no generaliza. Funciona en el parking cubierto y con luz artificial del profesorado (87%) y se desploma en exterior soleado (57%), hasta 43% en laterales. Esa es precisamente la justificación cuantitativa para pasar a YOLO.